2026 IMDb Prediction Challenge

Midterm Project

The MSE Avengers

Cameron Ku (261237196)

Melody Le (261103987)

Christina Liu (261222855)

Kevin Sun (261181689)

Samuel Yohai (261214560)

Faculty of Management, McGill University

MGSC 401-001: Statistical Foundations of Data

Professor Wenyu Jiao

March 10, 2026

1. Introduction (1-½ page)

In this project, our team (The MSE Avengers) builds a predictive model to estimate IMDb ratings for 12 upcoming movies in the provided test dataset. This project challenges us to predict IMDb ratings (1–10 scale, continuous) for 12 Winter 2026 blockbusters using a pre-cleaned dataset of historical movies. We use a training dataset of 1,930 movies with known IMDb scores and a wide set of predictors spanning film characteristics (e.g., budget, duration, genres), production information and details, cast popularity measures and metrics (e.g., top actors' star meters), release timing variables, and genre indicators. Our target is the IMDb score, which stabilizes post-release despite early fluctuations; therefore, our training data includes identifiers and both numeric and categorical predictors, and we generate predictions for the holdout test set of 12 films with no reliance on preliminary scores.

Using the provided dataset, we forecast these blockbusters to see if viewers will love or hate them. To achieve this, our approach follows the course workflow: we first understand the data and train a model based on initial assumptions, iterating as abnormalities arise. We begin with exploratory data analysis to understand variable distributions, identify potential transformations, and check relationships between predictors and IMDb score. Next, we compare candidate model specifications, applying linear and multiple regression along with non-linear transformations (including polynomials and splines) and selected interactions to create a flexible yet adequate model. We also use diagnostics to assess model assumptions and identify potential issues.

The goal is substantial confidence in our 12 predictions, balancing good reasoning with simplicity and prioritizing out-of-sample performance (e.g., low mean squared error) to avoid overfitting. Accordingly, we evaluate performance using out-of-sample methods such as a validation split and cross-validation to reduce overfitting risk, then select a final model based on predictive accuracy and interpretability. This process begins with data exploration and predictor rankings, followed by implementing non-linear terms and interactions, rigorous diagnostics, and model selection. Finally, we use the selected model to generate predicted IMDb scores for the 12 test movies.

The remainder of the report is organized as follows: Section 2 describes the dataset and key variables, Section 3 explains the model-building and selection process, and Section 4 presents the final model results and predicted IMDb scores for the test movies.

2. Data Description (2 - 3 pages)

Before diving into modelling, we first need to describe the dataset and summarize the preprocessing steps used to prepare the variables for analysis. These steps include basic cleaning, targeted transformations for skewed predictors, and feature selection to reduce unnecessary complexity and help the final model remain stable and interpretable.

2.1 IMDb Score (Target Variable)

The dependent variable in our model is the IMDb score, measured on a continuous 1-10 scale. Figure 1 shows the distribution of IMDb scores in the training set. Scores are concentrated in the mid-to-high range, with a mean of 6.512 and a median of 6.6, indicating that most movies receive moderate ratings. The distribution is slightly left-skewed, meaning relatively few films receive very low scores compared to the large number clustered around 6-7.

2.2 Movie Budget

Movie budget represents the total amount of money spent producing the film and is recorded as a continuous variable. The budget distribution (Figure 2a) is strongly right-skewed, showing that most films have smaller budgets while a smaller number have very large budgets. The scatterplot of IMDb score versus budget (Figure 2b) does not show a clear linear trend, but it does suggest that extremely high-budget films rarely appear among the lowest-rated movies. Because of the skewness, budget was considered a candidate for transformation (e.g., log scale) during model testing.

2.3 Release Year

Release year captures when the film debuted. While we explored this variable during EDA, it is less appealing for predicting future movies, since our goal is to generalize to new releases rather than explain historical time trends. As a result, release year was treated as a lower-priority predictor and considered for exclusion unless it improved out-of-sample performance.

2.4 Release Month

Release month is a categorical variable indicating the month of release. Figure 3a shows the number of movies released in each month. In this dataset, January and October have the largest counts, but the month-by-month IMDb score distributions (Figure 3b) show no strong systematic differences. Therefore, release month was treated as a potential predictor but not expected to be a primary driver of ratings.

2.5 Number of Faces

The number of faces is the number of actors/actresses detected on the movie poster. Its distribution (Figure 4) is right-skewed, indicating most posters feature relatively few faces. This variable was retained as a candidate predictor since it may proxy for marketing style or ensemble casts.

2.6 Duration

Film duration (minutes) emerged as one of the most informative predictors. The boxplot of duration is shown in Figure 5a, and Figure 5b displays the relationship between duration and IMDb score. Longer-running movies tend to receive higher ratings on average, and duration showed the strongest numeric correlation with IMDb score (approximately **r = 0.41**) in our correlation screening. Because the relationship appeared nonlinear, we explicitly tested nonlinear functional forms (e.g., quadratic terms and splines) during model selection. In this dataset, the mean film duration is **109.7 minutes** and the median is **106 minutes**; the middle 50% of films fall roughly between **90 and 120 minutes**, indicating that most movies cluster around typical feature-length runtimes.

2.7 Genre

Genres are represented with indicator variables (e.g., action, drama, war). Figure 6 compares IMDb scores across genres. We observe variability in median ratings by genre, with War showing higher typical ratings than many other genres. These indicators were retained because they provide interpretable differences in expected ratings across film types.

2.8 Distributor

The distributor variable has very high cardinality (approximately 1,000 unique distributors). To reduce complexity and avoid sparse dummy variables, we focused on the most frequent distributors. Figure 7 shows the top distributors by number of films, and the IMDb score distributions for the top 10 distributors show some variability, with **Miramax** having the highest median among this subset. In modeling, distributors were either limited to major categories (top distributors + "Other") or excluded depending on predictive value.

2.9 Cinematographer and Production Company

The cinematographer and production company each contain hundreds of unique values (high cardinality). Encoding them directly would create a large number of sparse categories and increase

overfitting risk. Because these variables add complexity without clear predictive benefit, we excluded them from the final modeling set.

2.10 Number of News Articles

Another important variable in our model is the number of news articles about the film. As seen in the boxplot (Figure 14), most movies receive a moderate amount of news articles in their respective countries with a median of 286. However, the mean amount of 770.6 is heavily inflated by extremely high outliers representing massive blockbuster movies that have thousands of articles written about them. Still, this variable is very useful as an interaction variable to duration in our final model.

2.11 Actor Star Meters

The actor star meters are IMDb's personalized rankings of actors/actresses where a lower rating represents higher popularity. As expected, the main actor had a greater median (1888) star meter than the secondary actor (3986), and the third actor (5856). This shows that the main actor is usually more popular than the secondary and tertiary star. The boxplot shows the variations and outliers for all three actor star meters (Figure 15).

3. Model Selection (1/2 - 1 page)

Model construction prioritized out-of-sample predictive power through iterations of univariate screening, functional form testing, and 10-fold cross-validation paired with validation set evaluation, with the dataset split 80/20 into training/validation sets using set.seed(1) for reproducibility.

Duration, the top univariate predictor (simple $R^2 = 0.169$), was tested via ANOVA, where quadratic significantly outperformed linear ($\Delta R^2 = +0.06$, $p < 0.001$) while cubic/quartic terms added no value ($p > 0.05$). Splines (quantile knots at 25/50/75%, degrees 1–4) were rejected despite slight in-sample gains, as validation RMSE went up by 0.9% relative to the quadratic, which violated parsimony. Logs for nb_news_articles underperformed raw forms ($R^2 = 0.041$ vs. 0.051) and were dropped.

Predictors were prioritized by univariate $R^2 > 0.03$ plus multivariate gains. release_year, nb_news_articles, movie_budget, and movie_meter_IMDBpro (all $p < 0.01$) were retained, as were 13 genre dummies (m3 validation MSE fell 3.2% compared to m2), and categoricals like maturity_rating and colour_film; high-cardinality language/country were collapsed to "Other" (min_count = 10). Predictors like plot_keywords and director were excluded due to VIF > 15, while release_day and aspect_ratio showed $p > 0.15$ and $\Delta R^2 < 0.001$.

Collinearity was acceptable with max VIF = 10.14 (nb_news_articles via interaction, below 15 threshold). Heteroskedasticity was confirmed by BP test ($p < 2.2e-16$) and the White test ($p = 4.3e-09$), but validation RMSE remained robust. Seven outliers were flagged but retained as valid observations to avoid bias. Overfitting occurred with train R^2 (0.413) exceeding validation (0.318), yet the model beat simplifications on validation MSE, justifying complexity. Underfitting was ruled out, as baseline quadratic MSE was 1.02 versus the final 0.855.

Figure 8 shows pairwise correlations among the main numeric predictors and imdb_score, which suggested no severe linear dependence among predictors. We then assessed multicollinearity more formally in the fitted model using variance inflation factors (VIF), and found it to be acceptable (max VIF = 10.14, driven by nb_news_articles through the interaction term and below our threshold of 15). We also tested model assumptions: heteroskedasticity was indicated by both the Breusch–Pagan test ($p < 2.2e-16$) and a White-style test ($p = 4.3e-09$), though validation RMSE remained stable. Seven observations were flagged by outlier diagnostics but were retained as valid cases to avoid bias. Finally, we monitored overfitting by comparing train and validation performance: train R^2 (0.413) exceeded

validation R^2 (0.318), but the final model still improved validation MSE relative to simpler alternatives (0.855 vs. 1.02 for the baseline quadratic), suggesting the added complexity was justified.

4. Results (2 - 3 pages)

Present the result for your final model. Tell us about your predictions for the 12 movies, the r-square of the model, the predictive power (i.e., out-of-sample performance), and the significance of each predictor.

Final Model Overview

The final model selected is a centered interaction regression "int_centered", incorporating a quadratic duration term, an interaction between film duration and media coverage, key numerical predictors, genre dummies, and categorical controls for country, language, maturity rating, and film colour. The quadratic aspect of "duration" was retained over a linear or spline form based on ANOVA testing in the model-building phase, which confirms that the quadratic term added statistically significant explanatory power ($p < 0.001$) while higher-degree polynomials produced negligible gains. The "duration × nb_news_articles" interaction was introduced to capture an asymmetry: media buzz may drive audience engagement differently for short, commercial films than for long, prestige productions. The data confirmed this, with the interaction term significant at the 1% level.

The model achieves a training R^2 of 0.413 and an adjusted R^2 of 0.393, explaining roughly 40% of the variance in IMDb scores, which is a meaningful result given the subjective nature of audience ratings. The overall F-statistic (25.22, $p < 2.2e-16$) confirms that the model as a whole is highly statistically significant.

Out-of-Sample Performance

A primary concern in model construction was avoiding overfitting. Candidate models were first ranked using "10-fold cross-validation" on the training set, then evaluated against a held-out "validation set" (20% of the labelled data, $n \approx 386$ observations). As shown in Figure 9 (in Appendices), the "int_centered" model consistently dominated all alternatives across both metrics:

The validation MSE of 0.855 exceeds 115% of the training MSE (0.704), which is technically a flag of overfitting. However, since `int_centered` still outperformed every simpler alternative on the validation set, no simplification was justified. In fact, simplifying the model would have *increased*

out-of-sample error, not reduced it. This suggests the gap between train and validation MSE reflects natural variance in the data rather than problematic overfitting.

The final model, refit on the combined training and validation set, achieves a "validation RMSE of 0.925" and "MAE of 0.663". This means predictions deviate from true IMDb scores by less than two-thirds of a rating point on average (refer to Figure 10 in the Appendices).

Predictor Significance & Interpretation

Rather than enumerating all coefficients, the discussion below focuses on the most economically and statistically meaningful findings (see Figure 12 for the full coefficient table).

"Film duration" exerts a positive but diminishing effect on IMDb scores. Longer films are rated more favourably, but extreme runtimes are penalized. This is in line with the intuition that lengthy films can suffer from pacing issues that lower audience satisfaction.

"Media coverage" (nb_news_articles) has a positive and significant effect, but its interaction with duration reveals an important nuance: the rating boost from media buzz is weaker for long films. One way to understand this can stem from the reasoning that media coverage can raise expectations that longer films have more trouble meeting, causing dissatisfaction

"Genre" is among the most powerful categorical predictors. Animation films score nearly +1.0 IMDb point higher on average. This reflects the strong critical reception of prestige animated features in the dataset. Drama adds +0.32 points, while action and horror are penalized by −0.35 and −0.37 points, respectively, consistent with the general perception that these genres trade critical praise for commercial appeal.

"Film colour" yields a surprising result: colour films score 0.5 points lower than black-and-white films on average, holding all else constant. This likely reflects survivorship bias, as older black-and-white films that remain in the dataset tend to be canonical classics with unusually high ratings. Therefore, interpretation should keep this aspect in mind.

"Cast star power" (actor1_star_meter, actor2_star_meter, actor3_star_meter) is notably "not significant" at conventional levels. This suggests that once genre, budget, and media coverage are set,

the fame of individual actors adds little independent predictive power. This is rather counterintuitive to the general philosophy that the cast is the main drivers to box office success.

“Release year” carries a small but significant negative coefficient (−0.018), indicating that more recent films tend to score marginally lower. This reflects recency bias in ratings since audiences may rate older films that have survived the test of time more generously. There is also the factor of nostalgia that can be considered for the rating of older films.

Predictions for the 12 Movies

The final model was applied to the 12 held-out films, producing the predictions in Figure 11.

Predictions fall within a range of 4.67 to 6.65, focused in the 5.3 to 6.4 band. A rather conservative range that reflects the model's tendency to regress toward the mean of the training distribution. The highest prediction goes to “My Love Will Make You Disappear” (6.65), driven in part by its lower-profile cast (reflected in high star meter values indicating lesser-known actors) and its Filipino-language, romance genre classification, which the model associates with niche audience appeal. The Day the Earth Blew Up (6.36) ranks second, benefiting from its animation genre tag, which the model consistently rewards. At the other end, Snow White (4.67) and Locked (5.28) score lowest. Snow White's prediction is notably suppressed despite its massive $250M budget, likely because its adventure/family genre profile and very high movie_meter_IMDBpro value interact unfavourably with the model's learned coefficients. Locked similarly reflects the model's penalization of the horror/thriller genre combination with limited news coverage.

The model's predictions are conservative, and the MAE of 0.663 on the validation set suggests individual predictions could realistically fall within ±0.7 of the true score. This is particularly relevant here given how concentrated the predictions are. 10 of the 12 films fall between 5.27 and 6.17, which means the rankings across close-scoring films should be interpreted with caution rather than as absolute.

5. Appendices (max 10 pages)

Figure 1: Distribution of IMDb score

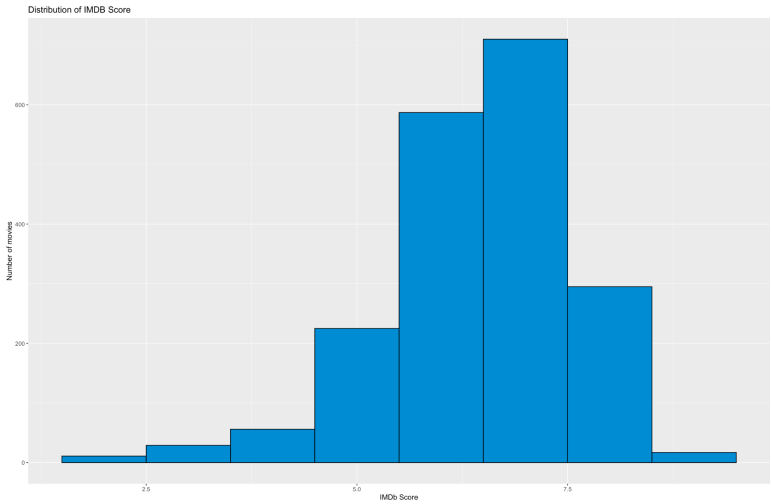
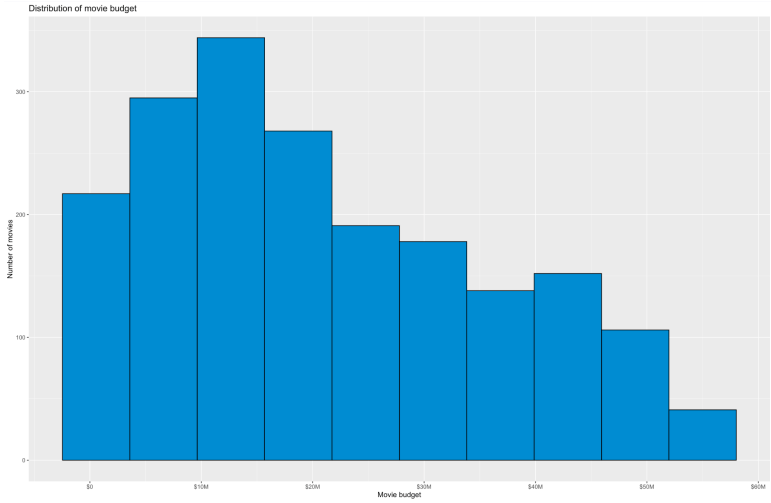


Figure 2: Movie budget

(2a) Distribution of movie budget



(2b) Distribution of movie budget

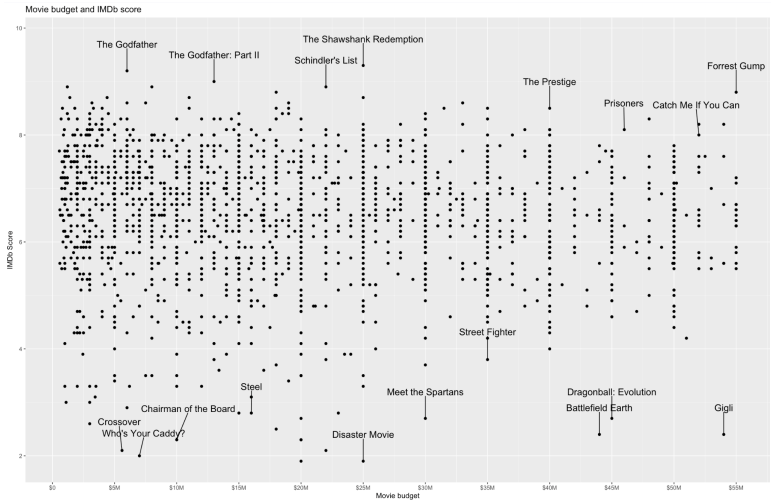
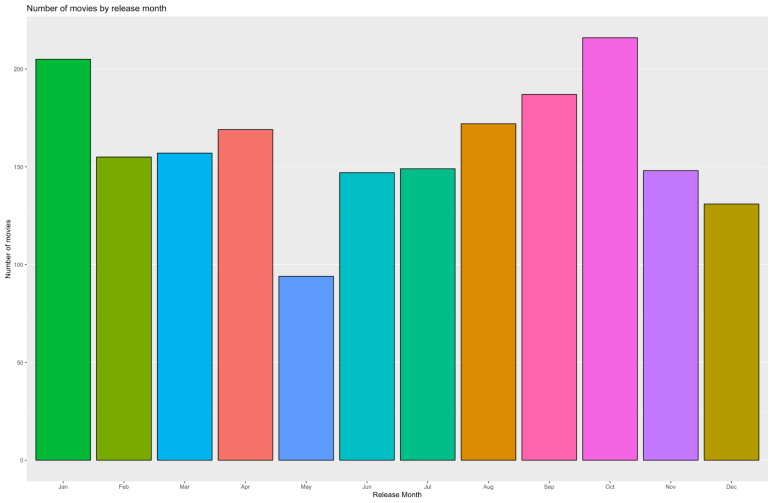


Figure 3: Release month

(3a) Number of movies by release month



(3b) Distribution of ratings across release month

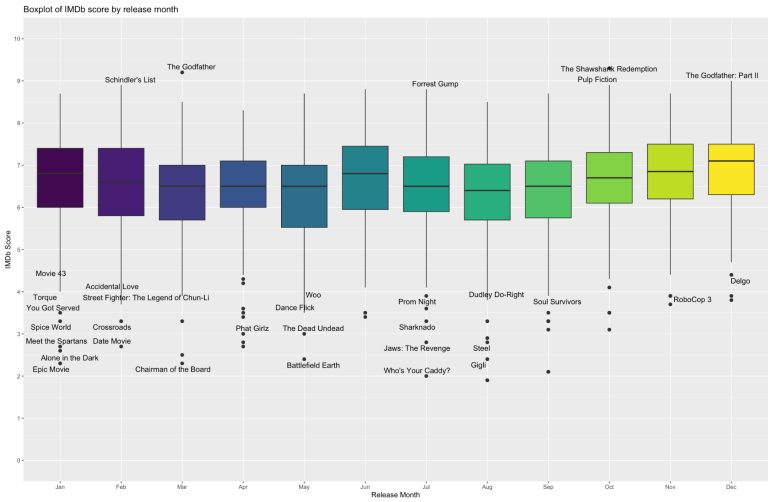


Figure 4: Distribution of number of faces

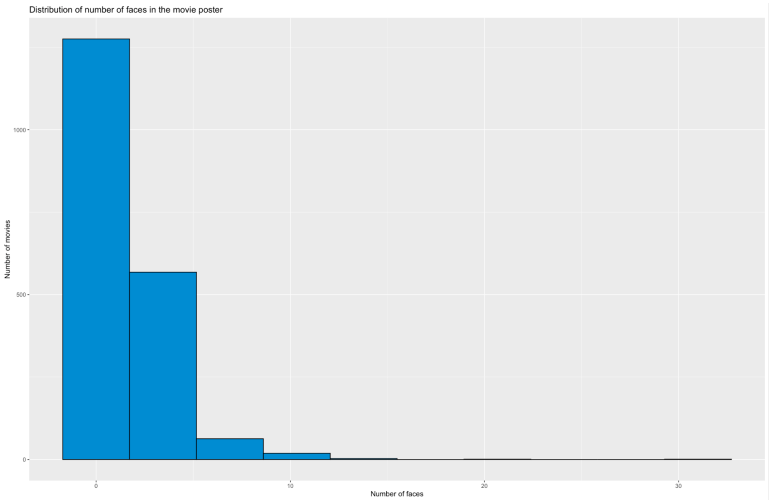
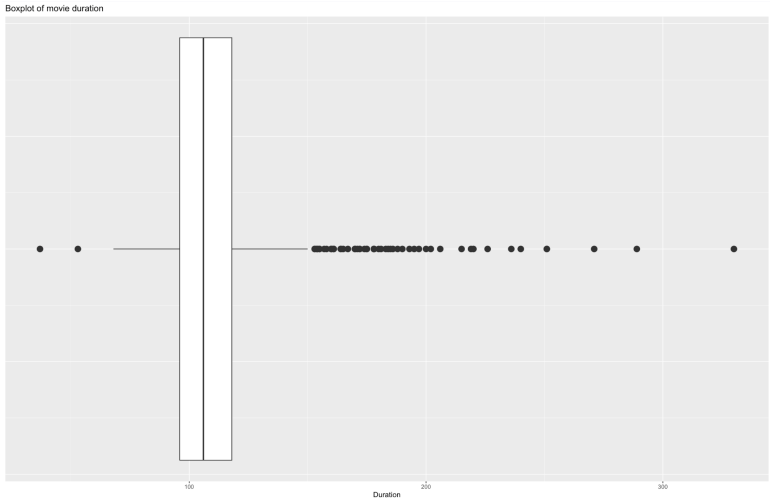


Figure 5: Duration

(5a) Boxplot of movie duration



(5b) IMDb score vs duration

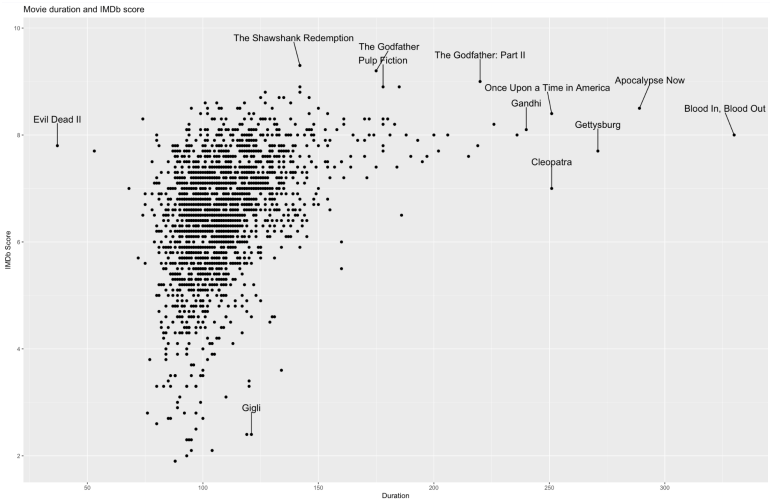


Figure 6: Distribution of scores across different genres

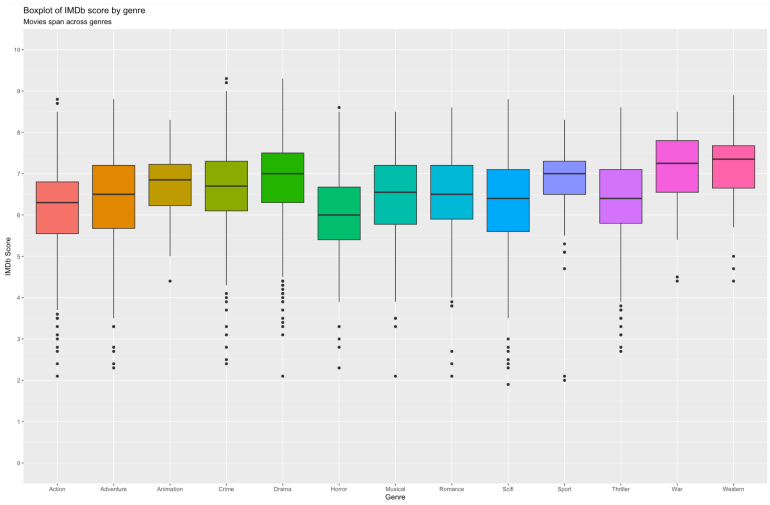


Figure 7: Distribution of movies across distributors

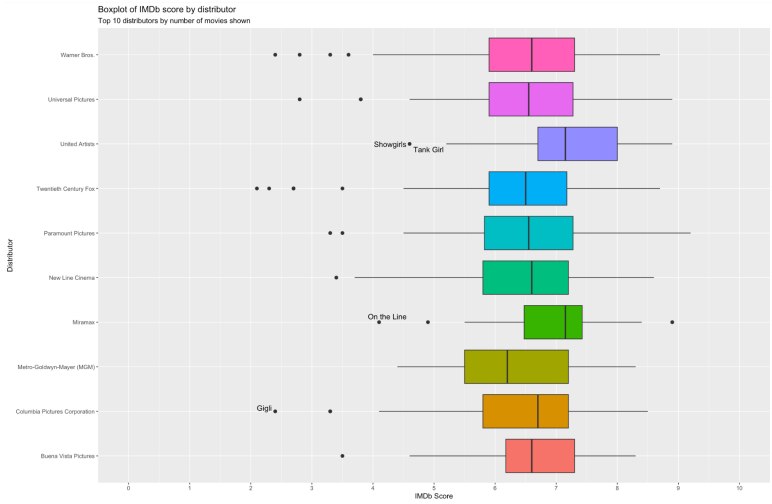


Figure 8: Correlation Matrix

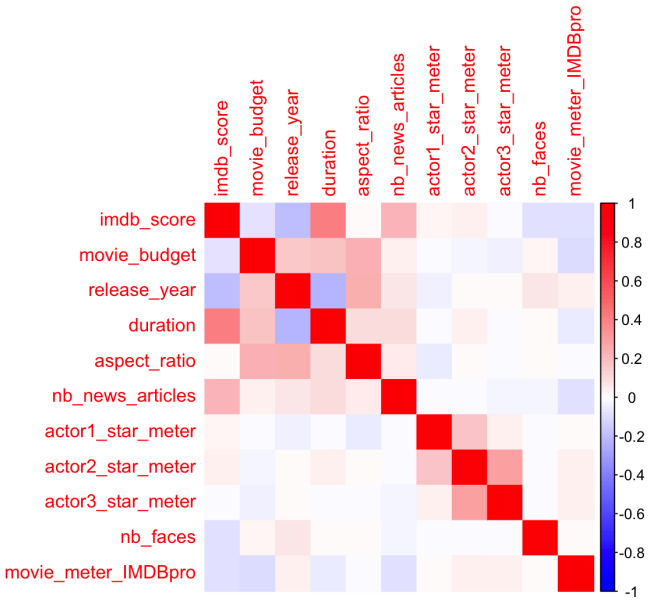


Figure 9: Model Comparison by Out-of Sample Performance

Model	Train MSE	CV MSE(10-fold)	Validation MSE
int_centered	0.704	0.857	0.855
m4 with categoricals	0.722	0.890	0.872
m3 with genres	0.769	0.923	0.915
m2 numeric only	0.865	0.933	0.977
m1 quadratic only	0/973	0..976	1.022

Figure 10: Final Model Performance Summary

Dataset	MSE	RMSE	MAE	R^2
Training set	0.704	0.839	0.612	0.413
Validation set	0.855	0.925	0.663	0.318
Train + Validation (refit)	0.724	0.851	0.622	0.402

Figure 11: IMDb Score Predictions for the 12 Target Movies

Movie Title	Predicted IMDb Score
O'Dessa	5.58
Black Bag	5.56
High Rollers	5.60
Novocaine	5.74
The Day the Earth Blew Up	6.36
Ash	5.40
Locked	5.28
Snow White	4.67
The Alto Knights	6.17
A Working Man	5.82
My love will make you disappear	**6.65**
The Woman in the Yard	5.27

Figure 12: Stargazer

Model Comparison: Build-Up to Final Model

	Dependent variable:				
	IMDb Score		paste("imdb_score ~ duration + duration2 + release_year + nb_news_articles +", "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter + actor3_star_meter +", "nb_faces + aspect_ratio + release_day +", paste(genre_dummies, collapse = " + "))	paste("imdb_score ~ duration + duration2 + release_year + nb_news_articles +", "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter + actor3_star_meter +", "nb_faces + aspect_ratio + release_day + release_month + language + country + maturity_rating + colour_film +", paste(genre_dummies, collapse = " + "))	imdb_score
	Quadratic	Numeric	With Genres	With Categoricals	Final (Interaction)
	(1)	(2)	(3)	(4)	(5)
duration	0.050***	0.051***	0.039***	0.035***	0.039***
	(0.005)	(0.005)	(0.005)	(0.005)	(0.005)
duration2	-0.0001***	-0.0001***	-0.0001***	-0.0001***	-0.0001***
	(0.00002)	(0.00002)	(0.00002)	(0.00002)	(0.00001)
release_year		-0.008***	-0.015***	-0.016***	-0.017***
		(0.002)	(0.002)	(0.003)	(0.002)
nb_news_articles		0.0001***	0.0001***	0.0001***	0.001***
		(0.00001)	(0.00001)	(0.00001)	(0.0001)
movie_budget		-0.000***	-0.000***	-0.000**	-0.000**
		(0.000)	(0.000)	(0.000)	(0.000)
movie_meter_IMDBpro		-0.00000**	-0.00000***	-0.00000***	-0.00000**
		(0.00000)	(0.00000)	(0.00000)	(0.00000)
nb_faces		-0.047***	-0.048***	-0.043***	-0.042***
		(0.012)	(0.011)	(0.011)	(0.011)
aspect_ratio		0.029	0.176	0.157	0.144
		(0.093)	(0.091)	(0.090)	(0.089)
release_day		0.003	0.002	0.002	0.001
		(0.003)	(0.003)	(0.003)	(0.003)
release_monthAug				0.030	0.052
				(0.107)	(0.106)
release_monthDec				0.114	0.072
				(0.118)	(0.117)
release_monthFeb				0.117	0.109
				(0.110)	(0.109)
release_monthJan				0.050	0.057
				(0.104)	(0.103)

release_monthJul				0.050	0.046
				(0.112)	(0.111)
release_monthJun				0.075	0.052
				(0.112)	(0.111)
release_monthMar				-0.006	-0.015
				(0.108)	(0.106)
release_monthMay				-0.245	-0.212
				(0.125)	(0.124)
release_monthNov				0.159	0.161
				(0.113)	(0.112)
release_monthOct				0.018	0.004
				(0.105)	(0.103)
release_monthSep				-0.042	-0.044
				(0.107)	(0.105)
maturity_ratingG				0.029	0.101
				(0.304)	(0.300)
maturity_ratingOther				-0.129	-0.176
				(0.300)	(0.297)
maturity_ratingPG				0.125	0.177
				(0.244)	(0.241)
maturity_ratingPG-13				0.068	0.119
				(0.247)	(0.244)
maturity_ratingR				0.362	0.411
				(0.243)	(0.241)
colour_filmColor				-0.487***	-0.497***
				(0.127)	(0.126)
action			-0.339***	-0.308***	-0.331***
			(0.068)	(0.067)	(0.066)
adventure			-0.187*	-0.136	-0.129
			(0.076)	(0.078)	(0.077)
scifi			0.052	0.039	0.015
			(0.081)	(0.080)	(0.079)
thriller			-0.109	-0.142*	-0.112
			(0.060)	(0.059)	(0.059)
musical			-0.297**	-0.262**	-0.262**
			(0.091)	(0.092)	(0.091)
romance			-0.138*	-0.090	-0.091
			(0.057)	(0.057)	(0.056)
western			0.144	0.139	0.123
			(0.173)	(0.171)	(0.169)
sport			0.057	0.125	0.153
			(0.113)	(0.113)	(0.112)
horror			-0.336***	-0.350***	-0.390***
			(0.082)	(0.082)	(0.081)
drama			0.357***	0.353***	0.334***
			(0.055)	(0.054)	(0.054)
war			0.153	0.033	0.043
			(0.129)	(0.127)	(0.126)
animation			1.068***	0.989***	1.070***
			(0.256)	(0.259)	(0.257)
crime			0.193**	0.158*	0.155*
			(0.064)	(0.065)	(0.064)
duration:nb_news_articles					-0.00001***
					(0.00000)
Constant	2.305***	18.143***	33.756***	35.739***	37.782***
	(0.324)	(4.523)	(4.545)	(4.943)	(4.893)
Country FE	No	No	No	Yes	Yes
Language FE	No	No	No	Yes	Yes
Interaction Term	No	No	No	No	Yes
Observations	1,544	1,544	1,544	1,544	1,544
R^2	0.188	0.279	0.359	0.398	0.413
Adjusted R^2	0.187	0.273	0.348	0.378	0.393
Note:					*p<0.05; **p<0.01; ***p<0.001

Figure 13: Spline with non-linear fit

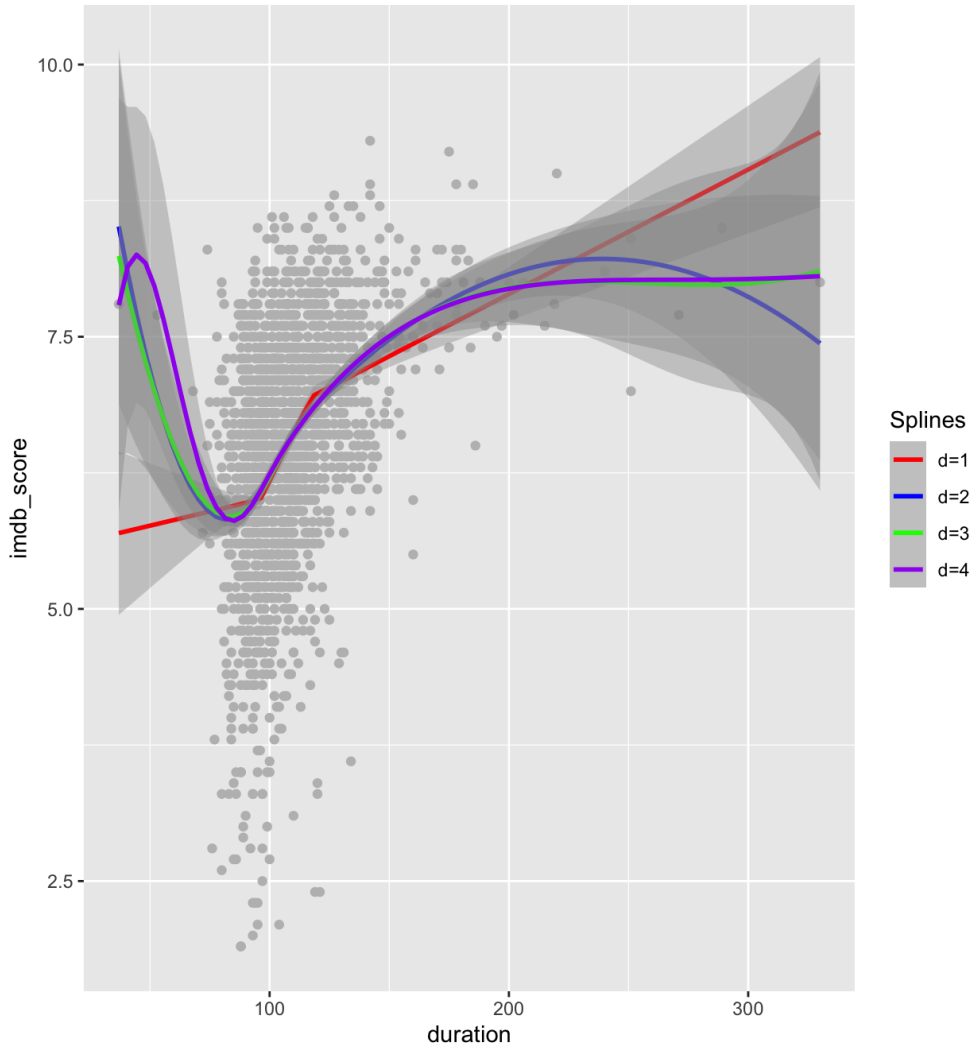


Figure 14: Boxplot for Number of Articles

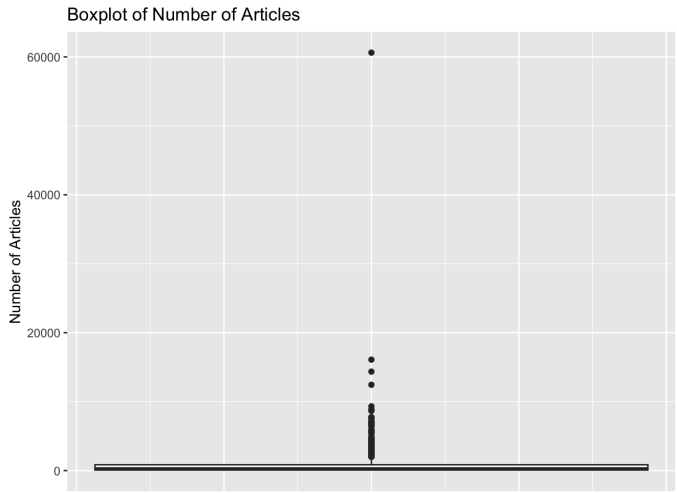
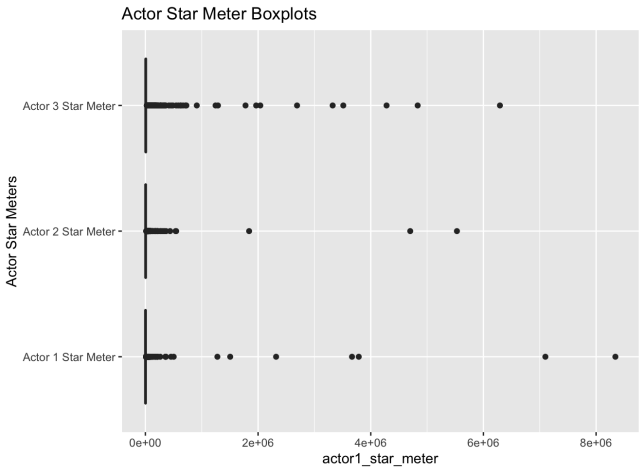


Figure 15: Boxplots for Actor Star Meters



Code

```
install.packages("e1071")
install.packages("corrplot")
install.packages("car")
install.packages("lmtest")
install.packages("plm")
install.packages("psych")
install.packages("splines")
install.packages("stargazer")

###---------- PLEASE CHANGE THE PATH TO YOUR OWN TO TEST THE CODE ----------###
IMDb_data = read.csv('/Users/kevinsun/MGSC 401/Midterm
Project/IMDB_data_Winter_2026.csv')
test_data = read.csv('/Users/kevinsun/MGSC 401/Midterm
Project/test_data_IMDB.csv')
data_dictionary = read.csv('/Users/kevinsun/MGSC 401/Midterm
Project/data_dictionary_IMDB_Winter_2026.csv')
View(IMDb_data)




################################################################################
###Part 1: Data Exploration
# ---------------------------------------------------------------------------
##a) Getting the "feel for the data set"
str(IMDb_data)
dim(IMDb_data)
summary(IMDb_data)
```

```
#Count missing value for each column
colSums(is.na(IMDb_data))

# ---------------------------------------------------------------------------
##b) Histogram & boxplots
#Select numerical variables
numeric_vars = IMDb_data[, c(
  "imdb_score",
  "movie_budget",
  "release_year",
  "duration",
  "aspect_ratio",
  "nb_news_articles",
  "actor1_star_meter",
  "actor2_star_meter",
  "actor3_star_meter",
  "nb_faces",
  "movie_meter_IMDBpro"
)]

#Plot All Histograms
par(mfrow = c(3, 4))  # 3x4 grid
for (var in names(numeric_vars)) {
  hist(numeric_vars[[var]],
       main = paste("Histogram of", var),
       col = "green",
       breaks = 20)
}

#Plot boxplots
par(mfrow = c(3, 4))
for (var in names(numeric_vars)) {
  boxplot(numeric_vars[[var]],
          main = paste("Boxplot of", var))
}

par(mfrow = c(1, 1)) #reset grid format

# ---------------------------------------------------------------------------
##c) Checking skewness
library(e1071)
skew_values = sapply(numeric_vars, skewness, na.rm = TRUE)
skew_values

# ---------------------------------------------------------------------------
##d) Correlation Matrix
```

```
cor_matrix <- cor(numeric_vars, use = "complete.obs")
round(cor_matrix, 2)
library(corrplot)
corrplot(cor_matrix,
         method = "color",
         col = colorRampPalette(c("blue", "white", "red"))(200))
#From the correlation matrix, find that "duration", "nb_news_articles"

# ---------------------------------------------------------------------------
##e) Outliers
full_model = lm(imdb_score ~ ., data = numeric_vars)
library(car)
outlierTest(full_model)
#Checking outliers
numeric_vars[c(492,1806,316,1581,191,989,395), ]
#Here we notice tha all 7 outliers are legitimate data points so they should not
be removed
#Result: no outlier removed

# ---------------------------------------------------------------------------
##f) Simple regressions for all numerical variables
for (var in names(numeric_vars)) {
  if (var != "imdb_score") {
    formula = as.formula(paste("imdb_score ~", var))
    model = lm(formula, data = IMDb_data)
    print(summary(model))
  }
}
#Based on the results from simple regressions, we have observed that duration
has the strongest R square (0.1686),
#, nb_news_articles the second strongest R square (0.05083), release_year the
third (0.03796)
#with the other variables with small to zero R square
#Therefore we will focus on these three variables for now

# ---------------------------------------------------------------------------
##g) Heteroskedasticity & Collinearity
#Heteroskedasicity
#Test with duration
library(lmtest)
library(car)
hreg = lm(imdb_score~duration, data = IMDb_data)
b0=coef(hreg)[1]
b1=coef(hreg)[2]
summary(hreg)
bptest(hreg)
```

```
#Funnel test
residualPlot(hreg, quadratic=FALSE)
#NCV test
ncvTest(hreg) #Note that p < 0.05 --> heteroskedasticity present
#Heteroskedasticity-corrected model
require(lmtest)
library(sandwich)
coeftest(hreg, vcov=vcovHC(hreg, type="HC1"))

#Collinearity
require(psych)
vif(full_model)
#Note that VIF for all variable are less than 4, there is no collinearity
problem present
quantvars = IMDb_data[,
c("imdb_score","duration","release_year","nb_news_articles","movie_budget")]
pairs.panels(quantvars,method = "pearson",hist.col = "lightgray",ellipses =
TRUE)
##############################################################################



##############################################################################
###Part 2: Functional Form & Non-Linearity
# ---------------------------------------------------------------------
## a)Testing Polynomial Terms
quadratic_reg1=lm(imdb_score ~ poly(duration, 2 ,raw = TRUE), data = IMDb_data)
#Degree 2
cubic_reg1=lm(imdb_score~ poly(duration,3, raw = TRUE), data = IMDb_data)
#Degree 3
quartic_reg1=lm(imdb_score~ poly(duration,4,raw = TRUE), data = IMDb_data)
#Degree 4
summary(hreg)$r.squared
summary(quadratic_reg1)$r.squared
summary(cubic_reg1)$r.squared
summary(quartic_reg1)$r.squared
anova(hreg, quadratic_reg1, cubic_reg1, quartic_reg1)
#Based on the results, quadratic model perform the best

# ---------------------------------------------------------------------
##b) Testing Log Transformations
log_model_duration= lm(imdb_score ~ log(duration), data = IMDb_data)
summary(log_model_duration)

log_model_news = lm(imdb_score ~ log(nb_news_articles + 1), data = IMDb_data)
summary(log_model_news)
```

```
log_model_year = lm(imdb_score ~ log(release_year), data = IMDb_data)
summary(log_model_year)

train_data = IMDb_data[idx_train, ]
valid_data = IMDb_data[-idx_train, ]  # save this immediately as valid_data
test_data_holdout = read.csv('/Users/kevinsun/MGSC 401/Midterm
Project/test_data_IMDB.csv')  # rename to avoid confusion

# ---------------------------------------------------------------------------
##c) Testing splines
library(splines)
library(ggplot2)

plot=ggplot(IMDb_data, aes(y=imdb_score, x=duration))
scatter= geom_point(color="grey")
knot_points = as.numeric(quantile(IMDb_data$duration, probs = c(0.25, 0.50,
0.75)))
# B-splines with explicit knots (degree 1 and degree 2)
spline_1= geom_smooth(method = "lm", formula = y~bs(x,knots=knot_points,
degree=1), aes(color="red"))
spline_2= geom_smooth(method = "lm", formula = y~bs(x,knots=knot_points,
degree=2), aes(color="blue"))
spline_3=geom_smooth(method = "lm", formula = y~bs(x,knots=knot_points,
degree=3), aes(color="green"))
spline_4=geom_smooth(method = "lm", formula = y~bs(x,knots=knot_points,
degree=4), aes(color="purple") )
spline_plot=plot+scatter+spline_1+spline_2+spline_3+spline_4
spline_plot
#To make a legend
spline_plot+scale_color_identity(name = "Splines",
                                 breaks = c("red", "blue", "green", "purple"),
                                 labels = c("d=1", "d=2", "d=3", "d=4"),
                                 guide = "legend")
#With quantiles
k1= quantile(IMDb_data$duration,.20)
k2= quantile(IMDb_data$duration,.40)
k3= quantile(IMDb_data$duration,.60)
k4= quantile(IMDb_data$duration,.80)
reg_w_knots=lm(imdb_score~bs(duration,knots=c(k1,k2,k3, k4), degree=3), data =
IMDb_data)
eq_spline= geom_smooth(method = "lm", formula = y~bs(x,knots=c(k1,k2,k3,k4),
degree=3))
plot+scatter+eq_spline+geom_vline(xintercept=c(k1,k2,k3,k4), linetype="dotted")

# ---------------------------------------------------------------------------
```

```
##d) Comparing R squared and performance
library(splines)
library(ggplot2)
set.seed(1)

n = nrow(IMDb_data)
idx_train = sample.int(n, size = floor(0.8 * n), replace = FALSE)

train_data = IMDb_data[idx_train, ]
test_data = IMDb_data[-idx_train, ]

rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
}

#Choose knot locations from train
knot_points = as.numeric(quantile(train_data$duration, probs = c(0.25, 0.50,
0.75), na.rm = TRUE))

# Fit models
m_linear    <- lm(imdb_score ~ duration, data = train_data)
m_quad      <- lm(imdb_score ~ poly(duration, 2, raw = TRUE), data = train_data)
m_cubic     <- lm(imdb_score ~ poly(duration, 3, raw = TRUE), data = train_data)
m_quartic   <- lm(imdb_score ~ poly(duration, 4, raw = TRUE), data = train_data)
m_log_dur   <- lm(imdb_score ~ log(duration), data = train_data)
m_log_news  <- lm(imdb_score ~ log(nb_news_articles + 1), data = train_data)
m_spline_d1 <- lm(imdb_score ~ bs(duration, knots = knot_points, degree = 1),
data = train_data)
m_spline_d2 <- lm(imdb_score ~ bs(duration, knots = knot_points, degree = 2),
data = train_data)
m_spline_d3 <- lm(imdb_score ~ bs(duration, knots = knot_points, degree = 3),
data = train_data)

# Collect R squared and RMSE
models <- list(
  linear      = m_linear,
  quadratic   = m_quad,
  cubic       = m_cubic,
  quartic     = m_quartic,
  log_duration= m_log_dur,
  log_news    = m_log_news,
  spline_d1   = m_spline_d1,
  spline_d2   = m_spline_d2,
  spline_d3   = m_spline_d3
)
```

```
results <- data.frame(
  model      = names(models),
  train_r2   = NA_real_,
  valid_rmse = NA_real_
)

for (i in seq_along(models)) {
  fit <- models[[i]]
  results$train_r2[i] <- summary(fit)$r.squared
  pred <- predict(fit, newdata = test_data)
  results$valid_rmse[i] <- rmse(test_data$imdb_score, pred)
}

# Sort by best validation performance (lowest RMSE)
results = results[order(results$valid_rmse), ]
print(results)

# ----------------------------------------------------------------------
## e)Decide linear V.S. non-linear model
best_model_name = results$model[1]
cat("\nBest model by validation RMSE:", best_model_name, "\n")
# Show the best model summary
cat("\nSummary of best model:\n")
print(summary(models[[best_model_name]]))
##############################################################################




##############################################################################
###Part 3: Build Model
# ----------------------------------------------------------------------
## a)Combining predictors into regression models
library(lmtest)
library(sandwich)
library(car)
library(stargazer)

#For Categorical Variables:
cat_vars = c("release_month","language","country","maturity_rating",
             "distributor","director","actor1","actor2","actor3",
             "colour_film","cinematographer","production_company")
for (v in cat_vars) {
  train_data[[v]] = as.factor(train_data[[v]])
  test_data[[v]] = as.factor(test_data[[v]])
}
```

```
for (v in cat_vars) {
  lv <- levels(train_data[[v]])
  test_data[[v]] = factor(test_data[[v]], levels = lv)
}

#For Numerical Variables:
num_base = c("movie_budget","release_day","release_year","aspect_ratio",

"nb_news_articles","actor1_star_meter","actor2_star_meter","actor3_star_meter",
             "nb_faces","movie_meter_IMDBpro")

#For Genre Variables (Dummified):
genre_dummies =
c("action","adventure","scifi","thriller","musical","romance","western",
                  "sport","horror","drama","war","animation","crime")

#Start with quadratic form of duration
train_data$duration2 = train_data$duration^2
test_data$duration2 = test_data$duration^2

#Baseline models
m1 = lm(imdb_score ~ duration + duration2, data = train_data)
m2 = lm(imdb_score ~ duration + duration2 +
          release_year + nb_news_articles +
          movie_budget + movie_meter_IMDBpro +
          actor1_star_meter + actor2_star_meter + actor3_star_meter +
          nb_faces + aspect_ratio + release_day,
        data = train_data)

#Model with genre dummies
m3 = lm(as.formula(paste(
  "imdb_score ~ duration + duration2 + release_year + nb_news_articles +",
  "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter +
actor3_star_meter +",
  "nb_faces + aspect_ratio + release_day +",
  paste(genre_dummies, collapse = " + "))), data = train_data)

#Model with categorical predictors
m4 = lm(as.formula(paste(
  "imdb_score ~ duration + duration2 + release_year + nb_news_articles +",
  "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter +
actor3_star_meter +",
  "nb_faces + aspect_ratio + release_day + release_month + language + country +
maturity_rating + colour_film +",
  paste(genre_dummies, collapse = " + ")
)), data = train_data)
```

```
#Compare Models
models_base = list(m1=m1, m2=m2, m3=m3, m4=m4)

base_results = data.frame(
  model = names(models_base),
  train_r2 = NA_real_,
  valid_rmse = NA_real_
)

for (i in seq_along(models_base)) {
  fit = models_base[[i]]
  base_results$train_r2[i] = summary(fit)$r.squared
  pred = predict(fit, newdata = test_data)
  base_results$valid_rmse[i] = rmse(test_data$imdb_score, pred)
}

base_results = base_results[order(base_results$valid_rmse), ]
print(base_results)

# ---------------------------------------------------------------------------
## b)Testing interactions & c)Handling dummy variables
#Side note: Rationale here: duration's effect on imdb score might varies
depending on
#           genre or rating, and news may matter more for newer films
m4_int1 = update(m4, . ~ . + duration:nb_news_articles)
m4_int2 = update(m4, . ~ . + duration:release_year)
m4_int4 = update(m4, . ~ . + nb_news_articles:release_year)
genre_int_terms = paste0("duration:", genre_dummies)
m4_int_genres = update(m4, as.formula(paste(". ~ . +", paste(genre_int_terms,
collapse = " + "))))

# Compare interaction models
models_int = list(
  m4 = m4,
  int1 = m4_int1,
  int2 = m4_int2,
  int_genres = m4_int_genres,
  int4 = m4_int4
)
int_results = data.frame(
  model = names(models_int),
  train_r2 = NA_real_,
  valid_rmse = NA_real_
)
for (i in seq_along(models_int)) {
```

```
  fit = models_int[[i]]
  int_results$train_r2[i] = summary(fit)$r.squared
  pred = predict(fit, newdata = test_data)
  int_results$valid_rmse[i] = rmse(test_data$imdb_score, pred)
}
int_results = int_results[order(int_results$valid_rmse), ]
print(int_results)

best_int_name = int_results$model[1]
best_model = models_int[[best_int_name]]

cat("\nBest model after interaction testing (by validation RMSE):",
best_int_name, "\n")
#from the code, we see that m4_int1 is the best model, in which we evaluate the
interation between
#duration (quadratic) and number of news articles, and therefore influence the
imdb_score

# ---------------------------------------------------------------------------
## d)Running diagnostics
cat("Best Model Summary:")
print(summary(best_model))
# Standard diagnostic plots
par(mfrow=c(2,2))
plot(best_model)
par(mfrow=c(1,1))
# Outliers / influence diagnostics
cat("Outlier Test")
print(outlierTest(best_model))

# ---------------------------------------------------------------------------
## e)Checking heteroskedasticity (White test, BP test)
#BP test
bp = bptest(best_model)
print(bp)

# White test
u2 = resid(best_model)^2
f = fitted(best_model)

white_proxy = lmtest::bptest(u2 ~ f + I(f^2))
print(white_proxy)

# ---------------------------------------------------------------------------
##f)Checking VIF for collinearity
best_model_fixed = lm(
```

```
  imdb_score ~ duration + duration2 +
    nb_news_articles +
    duration:nb_news_articles +
    release_year +
    movie_budget +
    movie_meter_IMDBpro +
    actor1_star_meter +
    actor2_star_meter +
    actor3_star_meter +
    nb_faces +
    aspect_ratio +
    release_day +
    maturity_rating +
    colour_film +
    action + adventure + scifi + thriller +
    musical + romance + western + sport +
    horror + drama + war + animation + crime,
  data = train_data
)
car::vif(best_model_fixed)

# ---------------------------------------------------------------------------
#The following section is for the final deliverable
#Stargazer tables
library(stargazer)
###stargazer(m1, m2, m3, m4,
          #type = "text",
          #title = "Regression Models (Core Build-Up)",
          #digits = 3,
          #omit.stat = c("f","ser"))

#Diagnostics summary
cat("Final model name:", best_int_name, "\n")
cat("Train R^2:", round(summary(best_model)$r.squared, 4), "\n")

pred_final <- predict(best_model, newdata = test_data)
cat("Validation RMSE:", round(rmse(test_data$imdb_score, pred_final), 4), "\n")

cat("\nBP test p-value:", format.pval(bp$p.value, digits=4), "\n")
cat("White proxy test p-value:", format.pval(white_proxy$p.value, digits=4),
"\n")

cat("\nShapiro test (residual normality) p-value:\n")
print(shapiro.test(residuals(best_model))$p.value)

library(stargazer)
```

```
stargazer(m1, m2, m3, m4, best_model,
          type = "text",
          title = "Model Comparison: Build-Up to Final Model",
          column.labels = c("Quadratic", "Numeric", "With Genres", "With
Categoricals", "Final (Interaction)"),
          dep.var.labels = "IMDb Score",
          digits = 3,
          omit.stat = c("f", "ser"),
          star.cutoffs = c(0.05, 0.01, 0.001),
          omit = c("country", "language", "director", "actor1", "actor2",
                   "actor3", "distributor", "cinematographer",
"production_company"),
          add.lines = list(
            c("Country FE",       "No", "No", "No", "Yes", "Yes"),
            c("Language FE",      "No", "No", "No", "Yes", "Yes"),
            c("Interaction Term", "No", "No", "No", "No",  "Yes")
          ),
          notes = "High-cardinality categorical variables (country, language,
etc.) omitted from display."
)


################################################################################




################################################################################
###Part 4: Predictions & Validations
# ----------------------------------------------------------------------
##a) K-fold or validation-set testing
# Candidate predictors
genre_dummies =
c("action","adventure","scifi","thriller","musical","romance","western",
                  "sport","horror","drama","war","animation","crime")

num_terms = c("movie_budget","release_day","release_year","aspect_ratio",

"nb_news_articles","actor1_star_meter","actor2_star_meter","actor3_star_meter",
              "nb_faces","movie_meter_IMDBpro")

# Candidate models
# 1) Simple quadratic on duration
f_m1 = imdb_score ~ duration + duration2
```

```
# 2) Numeric set + duration quadratic
f_m2 = imdb_score ~ duration + duration2 +
  release_year + nb_news_articles + movie_budget + movie_meter_IMDBpro +
  actor1_star_meter + actor2_star_meter + actor3_star_meter +
  nb_faces + aspect_ratio + release_day

# 3) With genre dummies
f_m3 = as.formula(paste(
  "imdb_score ~ duration + duration2 + release_year + nb_news_articles +",
  "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter +
actor3_star_meter +",
  "nb_faces + aspect_ratio + release_day +",
  paste(genre_dummies, collapse=" + ")
))

# 4) With key categorical variables
f_m4 = as.formula(paste(
  "imdb_score ~ duration + duration2 + release_year + nb_news_articles +",
  "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter +
actor3_star_meter +",
  "nb_faces + aspect_ratio + release_day +",
  "release_month + language + country + maturity_rating + colour_film +",
  paste(genre_dummies, collapse=" + ")
))

# 5) Centered best interaction idea: duration * nb_news_articles (with quadratic
duration)
f_int_centered = as.formula(paste(
  "imdb_score ~ c_duration + c_duration2 + release_year + c_nb_news +",
  "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter +
actor3_star_meter +",
  "nb_faces + aspect_ratio + release_day +",
  "release_month + language + country + maturity_rating + colour_film +",
  paste(genre_dummies, collapse=" + "),
  "+ c_duration:c_nb_news"
))

models_formulas = list(
  m1 = f_m1,
  m2 = f_m2,
  m3 = f_m3,
  m4 = f_m4,
  int_centered = f_int_centered
)

cat_vars =
```

```
c("release_month","language","country","maturity_rating","colour_film")

mse = function(actual, predicted) {mean((actual - predicted)^2, na.rm = TRUE)}

collapse_rare = function(x, min_count = 10) {
  x = as.character(x)
  tab = table(x)
  rare = names(tab[tab < min_count])
  x[x %in% rare] = "Other"
  factor(x)
}

for (v in cat_vars) {
  train_data[[v]] = collapse_rare(train_data[[v]], min_count = 10)
}

# K-fold CV function
kfold_cv_mse = function(formula, data, cat_vars, K = 10, seed = 1) {
  set.seed(seed)
  n = nrow(data)
  folds = sample(rep(1:K, length.out = n))
  master_levels = list()
  for (v in cat_vars) {
    master_levels[[v]] = levels(as.factor(data[[v]]))
  }

  fold_mse = numeric(K)

  for (k in 1:K) {
    tr = data[folds != k, , drop = FALSE]
    te = data[folds == k, , drop = FALSE]
    for (v in cat_vars) {
      tr[[v]] = factor(as.character(tr[[v]]), levels = master_levels[[v]])
      te[[v]] = factor(as.character(te[[v]]), levels = master_levels[[v]])
    }
    #Create centered variables inside the fold
    mean_dur  = mean(tr$duration, na.rm = TRUE)
    mean_news = mean(tr$nb_news_articles, na.rm = TRUE)
    tr$c_duration = tr$duration - mean_dur
    te$c_duration = te$duration - mean_dur
    tr$c_duration2 = tr$c_duration^2
    te$c_duration2 = te$c_duration^2
    tr$c_nb_news = tr$nb_news_articles - mean_news
    te$c_nb_news = te$nb_news_articles - mean_news

    fit = lm(formula, data = tr)
```

```
    pred = predict(fit, newdata = te)

    fold_mse[k] = mse(te$imdb_score, pred)
  }

  mean(fold_mse)
}

#Running CV on the candidate models
K = 10
cv_table = data.frame(
  model = names(models_formulas),
  train_mse = NA_real_,
  cv_mse = NA_real_
)

train_data$c_duration  = train_data$duration - mean(train_data$duration, na.rm =
TRUE)
train_data$c_duration2 = train_data$c_duration^2
train_data$c_nb_news   = train_data$nb_news_articles -
mean(train_data$nb_news_articles, na.rm = TRUE)

for (i in seq_along(models_formulas)) {
  f = models_formulas[[i]]
  fit_train = lm(f, data = train_data)
  pred_train = predict(fit_train, newdata = train_data)
  cv_table$train_mse[i] = mse(train_data$imdb_score, pred_train)
  cv_table$cv_mse[i] = kfold_cv_mse(f, data = train_data, cat_vars = cat_vars, K
= K, seed = 1)
}
cv_table = cv_table[order(cv_table$cv_mse), ]
print(cv_table)

# ---------------------------------------------------------------------------
##b) Comparing models by out-of-sample MSE
# Ensure test_data's factor levels to be the same as train_data


if (!exists("cat_vars")) stop("cat_vars not found. It should be defined in
section (a).")

for (v in cat_vars) {
  # Make sure both are character first (stable handling)
  train_data[[v]] = as.character(train_data[[v]])
  valid_data[[v]] = as.character(valid_data[[v]])
  lv = sort(unique(train_data[[v]]))
```

```
  if (!("Other" %in% lv)) lv = c(lv, "Other")
  train_data[[v]] = factor(train_data[[v]], levels = lv)
  x = valid_data[[v]]
  x[is.na(x)] = "Other"
  x[!(x %in% lv)] = "Other"
  valid_data[[v]] = factor(x, levels = lv)
}

# Build variables
train_data$duration2 = train_data$duration^2
valid_data$duration2 = valid_data$duration^2
mean_dur_train  = mean(train_data$duration, na.rm = TRUE)
mean_news_train = mean(train_data$nb_news_articles, na.rm = TRUE)
train_data$c_duration  = train_data$duration - mean_dur_train
valid_data$c_duration  = valid_data$duration - mean_dur_train
train_data$c_duration2 = train_data$c_duration^2
valid_data$c_duration2 = valid_data$c_duration^2
train_data$c_nb_news   = train_data$nb_news_articles - mean_news_train
valid_data$c_nb_news   = valid_data$nb_news_articles - mean_news_train

# Fit models on train_data
if (!exists("mse")) {
  mse = function(actual, predicted) mean((actual - predicted)^2, na.rm = TRUE)
}

oos_table = data.frame(
  model     = names(models_formulas),
  train_mse = cv_table$train_mse[match(names(models_formulas), cv_table$model)],
  cv_mse    = cv_table$cv_mse[match(names(models_formulas), cv_table$model)],
  oos_mse   = NA_real_
)

for (i in seq_along(models_formulas)) {
  f = models_formulas[[i]]
  fit = lm(f, data = train_data)
  pred_valid = predict(fit, newdata = valid_data)
  oos_table$oos_mse[i] = mse(valid_data$imdb_score, pred_valid)
}

# Rank models by out-of-sample MSE (lower is better)
oos_table = oos_table[order(oos_table$oos_mse), ]
print(oos_table)

# Identify best model by OOS MSE
best_oos_name = oos_table$model[1]
cat("Best model by out-of-sample MSE (evaluated on valid_data):", best_oos_name,
```

```
"\n")

# ----------------------------------------------------------------------
##c) Simplifying if overfitting occurs
#Rationale: If the best model outperforms on train than on test, then we
simplify
#           the model by dropping interation and/or high dimensional categorical
terms
# Rebuild duration2 for all datasets

best_name = oos_table$model[1]
best_formula = models_formulas[[best_name]]

#Compute train vs validation MSE for the best model
fit_best = lm(best_formula, data = train_data)
pred_train_best = predict(fit_best, newdata = train_data)
pred_valid_best = predict(fit_best, newdata = valid_data)
train_mse_best = mse(train_data$imdb_score, pred_train_best)
valid_mse_best = mse(valid_data$imdb_score, pred_valid_best)
cat("\nCurrent best model:", best_name, "\n")
cat("Train MSE:", round(train_mse_best, 4), "\n")
cat("Valid  MSE:", round(valid_mse_best, 4), "\n")

# Overfitting flag rule: model will be classified as "likely overfitting" if
#                        validation MSE is > 15% worse than train MSE
overfit_flag = (valid_mse_best > 1.15 * train_mse_best)
cat("Overfitting flag (Valid MSE > 1.15 * Train MSE):", overfit_flag, "\n")

# When overfitting problem present --> try simpler candidate models

if (overfit_flag) {
  cat("\nTrying simpler models because overfitting was flagged...\n")
  simplify_candidates = c("m1", "m2", "m3", "m4", "int_centered")
  simplify_candidates = simplify_candidates[simplify_candidates %in%
names(models_formulas)]
  simplify_table = data.frame(
    model = simplify_candidates,
    train_mse = NA_real_,
    valid_mse = NA_real_
  )
  for (i in seq_along(simplify_candidates)) {
    nm = simplify_candidates[i]
    f  = models_formulas[[nm]]
    fit = lm(f, data = train_data)
    p_tr = predict(fit, newdata = train_data)
    p_va = predict(fit, newdata = valid_data)
```

```
    simplify_table$train_mse[i] = mse(train_data$imdb_score, p_tr)
    simplify_table$valid_mse[i] = mse(valid_data$imdb_score, p_va)
  }
  simplify_table = simplify_table[order(simplify_table$valid_mse), ]
  print(simplify_table)
  # Update the best model
  best_name    = simplify_table$model[1]
  best_formula = models_formulas[[best_name]]
  cat("\nChosen simplified model (best by validation MSE):", best_name, "\n")
} else {
  cat("\nNo simplification applied (overfitting not flagged).\n")
}

# ---------------------------------------------------------------------------
##d) Generating final predictions for 12 movies
#Make sure categorical variables in train/test match
for (v in cat_vars) {
  train_data[[v]] = as.character(train_data[[v]])
  lv = sort(unique(train_data[[v]]))
  if (!("Other" %in% lv)) lv = c(lv, "Other")
  train_data[[v]] = factor(train_data[[v]], levels = lv)
  x = as.character(test_data[[v]])
  x[is.na(x)] = "Other"
  x[!(x %in% lv)] = "Other"
  test_data[[v]] = factor(x, levels = lv)
}

# Build needed numeric variables
# duration2
train_data$duration2 = train_data$duration^2
test_data$duration2  = test_data$duration^2
# Centering terms (MUST use train means)
mean_dur_train  = mean(train_data$duration, na.rm = TRUE)
mean_news_train = mean(train_data$nb_news_articles, na.rm = TRUE)
train_data$c_duration  = train_data$duration - mean_dur_train
test_data$c_duration   = test_data$duration  - mean_dur_train
train_data$c_duration2 = train_data$c_duration^2
test_data$c_duration2  = test_data$c_duration^2
train_data$c_nb_news = train_data$nb_news_articles - mean_news_train
test_data$c_nb_news  = test_data$nb_news_articles  - mean_news_train

# Fit final model
final_train = rbind(train_data, valid_data)
final_fit = lm(best_formula, data = final_train)

# Predict for the 12 movies
```

```
test_pred = predict(final_fit, newdata = test_data)

# Save predictions
predictions_12 = data.frame(
  movie_id = if ("movie_id" %in% names(test_data)) test_data$movie_id else NA,
  movie_title = if ("movie_title" %in% names(test_data)) test_data$movie_title
else NA,
  pred_imdb_score = test_pred
)

print(predictions_12)

# ----------------------------------------------------------------------
##e) Calculating final model performance metrics
#Define metrics
rmse = function(actual, predicted) sqrt(mean((actual - predicted)^2, na.rm =
TRUE))
mae  = function(actual, predicted) mean(abs(actual - predicted), na.rm = TRUE)
r2   = function(actual, predicted) {
  sse = sum((actual - predicted)^2, na.rm = TRUE)
  sst = sum((actual - mean(actual, na.rm = TRUE))^2, na.rm = TRUE)
  1 - sse/sst
}

# Predictions on train and valid using a model fitted on train only
fit_report = lm(best_formula, data = train_data)
pred_tr = predict(fit_report, newdata = train_data)
pred_va = predict(fit_report, newdata = valid_data)

# Create a performance table
perf_table = data.frame(
  dataset = c("train_data", "valid_data", "train+valid (refit)"),
  MSE  = NA_real_,
  RMSE = NA_real_,
  MAE  = NA_real_,
  R2   = NA_real_
)

# Train metrics
perf_table$MSE[1]  = mse(train_data$imdb_score, pred_tr)
perf_table$RMSE[1] = rmse(train_data$imdb_score, pred_tr)
perf_table$MAE[1]  = mae(train_data$imdb_score, pred_tr)
perf_table$R2[1]   = r2(train_data$imdb_score, pred_tr)

# Validation metrics
perf_table$MSE[2]  = mse(valid_data$imdb_score, pred_va)
```

```
perf_table$RMSE[2] = rmse(valid_data$imdb_score, pred_va)
perf_table$MAE[2]  = mae(valid_data$imdb_score, pred_va)
perf_table$R2[2]   = r2(valid_data$imdb_score, pred_va)

# Combined metrics (fit on final_train, predict on final_train)
pred_all = predict(final_fit, newdata = final_train)
perf_table$MSE[3]  = mse(final_train$imdb_score, pred_all)
perf_table$RMSE[3] = rmse(final_train$imdb_score, pred_all)
perf_table$MAE[3]  = mae(final_train$imdb_score, pred_all)
perf_table$R2[3]   = r2(final_train$imdb_score, pred_all)

#Print final performance
print(perf_table)
cat("\nFinal chosen model name:", best_name, "\n")
cat("Final chosen formula:\n")
print(best_formula)
cat("\nNote: test_data has no imdb_score, so performance metrics cannot be
computed on test_data.\n")
################################################################################

print(summary(final_fit))



#RESULT PREDICTIONS
#1 - file path
IMDb_data      <- read.csv('/Users/kevinsun/MGSC 401/Midterm
Project/IMDB_data_Winter_2026.csv')
test_data_raw  <- read.csv('/Users/kevinsun/MGSC 401/Midterm
Project/test_data_IMDB.csv')  # your 12-movie file

# ----------------------------------------------------------------------
# 2- re-run the train/valid split
set.seed(1)
n          <- nrow(IMDb_data)
idx_train  <- sample.int(n, size = floor(0.8 * n), replace = FALSE)
train_data <- IMDb_data[idx_train, ]
valid_data <- IMDb_data[-idx_train, ]

# ----------------------------------------------------------------------
# 3- define helper & categorical collapsing

cat_vars <- c("release_month", "language", "country", "maturity_rating",
"colour_film")

collapse_rare <- function(x, min_count = 10) {
```

```
  x   <- as.character(x)
  tab <- table(x)
  x[x %in% names(tab[tab < min_count])] <- "Other"
  factor(x)
}

for (v in cat_vars) {
  train_data[[v]] <- collapse_rare(train_data[[v]], min_count = 10)
}

# ----------------------------------------------------------------------
# 4- align factor levels

test_data <- test_data_raw   # work on a copy

for (v in cat_vars) {
  train_data[[v]] <- as.character(train_data[[v]])
  lv              <- sort(unique(train_data[[v]]))
  if (!("Other" %in% lv)) lv <- c(lv, "Other")
  train_data[[v]] <- factor(train_data[[v]], levels = lv)

  x        <- as.character(test_data[[v]])
  x[is.na(x)]        <- "Other"
  x[!(x %in% lv)]    <- "Other"   # unseen levels → "Other"
  test_data[[v]]      <- factor(x, levels = lv)
}

# ----------------------------------------------------------------------
# 5- build engineered features

train_data$duration2  <- train_data$duration^2
test_data$duration2   <- test_data$duration^2

mean_dur_train  <- mean(train_data$duration,         na.rm = TRUE)
mean_news_train <- mean(train_data$nb_news_articles,  na.rm = TRUE)

train_data$c_duration  <- train_data$duration         - mean_dur_train
test_data$c_duration   <- test_data$duration          - mean_dur_train
train_data$c_duration2 <- train_data$c_duration^2
test_data$c_duration2  <- test_data$c_duration^2
train_data$c_nb_news   <- train_data$nb_news_articles - mean_news_train
test_data$c_nb_news    <- test_data$nb_news_articles  - mean_news_train

# Also apply to valid_data for the final combined fit
valid_data$duration2  <- valid_data$duration^2
valid_data$c_duration <- valid_data$duration         - mean_dur_train
```

```
valid_data$c_duration2<- valid_data$c_duration^2
valid_data$c_nb_news  <- valid_data$nb_news_articles  - mean_news_train

for (v in cat_vars) {
  lv <- levels(train_data[[v]])
  x  <- as.character(valid_data[[v]])
  x[is.na(x) | !(x %in% lv)] <- "Other"
  valid_data[[v]] <- factor(x, levels = lv)
}

# ----------------------------------------------------------------------
# 6- Define the best formula (int_centered)

genre_dummies <- c("action","adventure","scifi","thriller","musical","romance",
                   "western","sport","horror","drama","war","animation","crime")

best_formula <- as.formula(paste(
  "imdb_score ~ c_duration + c_duration2 + release_year + c_nb_news +",
  "movie_budget + movie_meter_IMDBpro + actor1_star_meter + actor2_star_meter +
actor3_star_meter +",
  "nb_faces + aspect_ratio + release_day +",
  "release_month + language + country + maturity_rating + colour_film +",
  paste(genre_dummies, collapse = " + "),
  "+ c_duration:c_nb_news"
))

# ----------------------------------------------------------------------
# 7- fit the 'final' model on train + valid combined, then predict

final_train <- rbind(train_data, valid_data)
final_fit   <- lm(best_formula, data = final_train)

test_predictions <- predict(final_fit, newdata = test_data)

# ----------------------------------------------------------------------
# 8- results

predictions_df <- data.frame(
  movie_id        = test_data$movie_id,
  movie_title     = test_data$movie_title,
  pred_imdb_score = round(test_predictions, 2)
)

print(predictions_df)
write.csv(predictions_df, "imdb_predictions.csv", row.names = FALSE)
```